

DIGITAL TRANSFORMATION IN LENDING

BharatBanker AI

Unified Solution Document

AI-Powered Hyper-Personalized Banking for Bharat: a single decisioning platform for proactive product recommendation, vernacular-first conversational banking, and empathetic fraud and financial-stress detection.

Smart Recommendation
Engine

Vernacular Conversational
AI

Financial Health
Monitor

Hackathon Submission | September 2026

Synthesized from the solution deck, the technical architecture report,
the recommendation-engine algorithm note, and the application-security note.

This document is for hackathon evaluation purposes only.

1 . Executive Summary

BharatBanker AI is an intelligent banking companion that treats the three challenge sub-problems (proactive personalization, vernacular-first conversational access, and empathetic fraud/financial-stress detection) as **one connected decisioning platform** rather than three independent features. A shared behavioural feature pipeline, built once from a customer's transactions, EMI history and app behaviour, feeds three specialist model heads (product-propensity, anomaly detection, stress-trend detection). All three are gated through a single explainable **veto layer** before anything reaches the customer, so a stressed or flagged customer can never be pushed a loan offer on one channel while being flagged as at-risk on another.

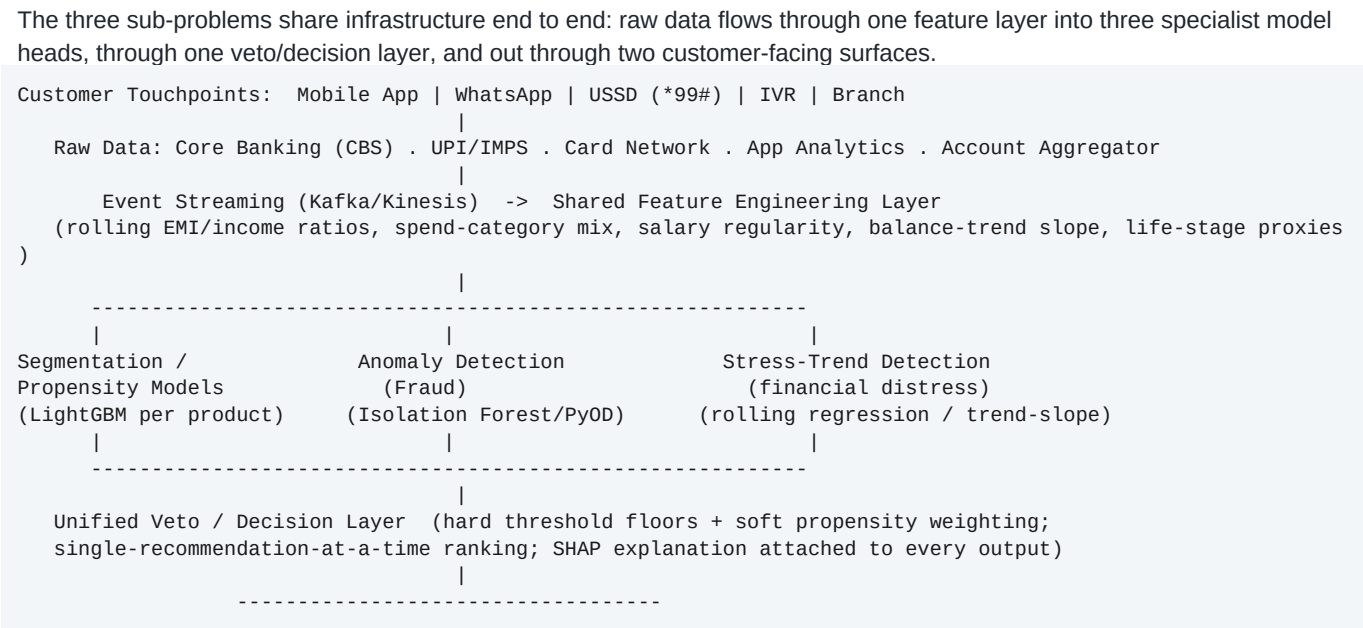
The differentiator is not model sophistication but **decision discipline**: every recommendation is explainable (SHAP-based, plain-language), every credit-product push is subject to a hard affordability/stress floor, not just a soft discount, and every vernacular conversation is grounded in cited regulatory/government sources rather than free-form generation. This targets the judging emphasis on *genuine customer benefit over upsell* and *regulatory compliance readiness*, alongside technical novelty.

Pillar	What it does	Core technique
Smart Recommendation Engine	Analyses transactions, spend, EMI and life-stage signals to surface the single most relevant product at the right moment.	Behavioural segmentation + per-product LightGBM propensity models, veto-gated
Vernacular Conversational AI	Simplifies onboarding, loan application and KYC through a multilingual chatbot (Hindi, Tamil, Telugu, Marathi, Bengali and more).	Slot-filling task engine + multilingual RAG, routed by intent
Financial Health Monitor	Detects early signals of financial stress or fraud and triggers supportive, non-punitive interventions.	Isolation Forest (fraud) + trend-slope regression (stress), same veto layer

Design Principles

- **One shared feature pipeline, three specialist model heads**, not one monolith. Personalization, fraud and stress detection read different signal types (segment-level, point-anomaly, trend) from the same transaction data; forcing one model to serve all three would blur three distinct objectives.
- **Explainability by construction**: every recommendation carries a SHAP-based justification and every RAG answer a citation, generated as part of the output format, not assembled after the fact for judges.
- **Hard floors beneath soft weighting**: wherever a decision could cause real financial harm, a deterministic rule-based veto sits beneath the ML-driven ranking. ML tunes the experience; it does not have unlimited authority over risk-sensitive actions.
- **Purpose-bound data, not maximal profiling**: customer context is tiered by consent purpose (implicit account data, then opt-in behavioural data, then explicit life-stage data), aligned to the DPDP Act's purpose-limitation principle.
- **One recommendation at a time**: the system deliberately avoids stacking multiple product offers, directly countering the generic multi-offer pop-up pattern the problem statement flags as the status-quo failure.

2 . Unified System Architecture



Personalized Dashboard
(Streamlit -> Next.js)

Vernacular Chatbot (Slot-Filling for
KYC/loan journeys + Multilingual RAG for
policy Q&A; grounded in RBI/govt sources)

End-to-End Data Flow

- **Collect:** customer transacts (UPI payment, EMI debit, salary credit); event streams to Kafka in real time.
- **Screen:** real-time anomaly check runs first; a flagged event triggers the intervention workflow immediately, bypassing the recommendation path.
- **Engineer:** Customer 360 features update (spend pattern, balance trend, life-stage score).
- **Infer:** the relevant model head(s) run inference (<200ms for lightweight edge models; heavier batch models refresh every 4 to 6 hours).
- **Decide:** the veto layer applies hard floors and soft weighting, checks eligibility, affordability and contact frequency, and ranks the single best action.
- **Deliver / Intervene:** pass leads to delivery via the customer's preferred channel and language; fail is logged and the customer is not contacted; stress or fraud is routed to the empathetic-intervention or fraud-hold flow instead.

3 . Sub-Problem 1: Hyper-Personalized Recommendation Engine

Approach. Pure collaborative filtering was rejected because it fails on first-time digital users with sparse interaction history (the cold-start problem this challenge explicitly targets). Instead we use a **two-layer approach**: behavioural segmentation and rule-based life-stage detection, followed by per-product propensity ranking, gated by the affordability/stress veto layer.

Consent-Scoped Data Tiers

Tier	Data	Consent basis	Used for
Tier 0: Implicit	Salary credit regularity, account vintage, KYC occupation	Account relationship	All products
Tier 1: Behavioural	EMI history, spend-category mix, savings velocity, balance trend	Opt-in at onboarding	Segmentation, propensity scoring
Tier 2: Explicit	Life-stage signals (marriage, home-purchase intent, dependents)	Per-purpose opt-in	Investment, insurance only

This tiering is deliberate: one maximal profile object would violate DPDP's purpose-limitation principle and raise proxy-discrimination risk. Every recommendation is traceable to the tier that justified it.

Behavioural Segmentation: K-Means on Engineered Features

Customers are clustered on engineered ratios (disposable income, savings ratio, debt-to-income, balance volatility, category spend vector) using **K-Means with silhouette-score optimisation** (k = 3 to 12, PCA if more than 15 features). This surfaces segments such as Young Earners, Stable Professionals, Family Builders, High Net-worth and, critically, a **Financially Stressed** segment that is excluded from credit-product targeting by design, not by afterthought.

A parallel rule layer catches temporal life-events that clustering alone would miss and average out; this is what makes recommendations feel timely rather than generic:

Signal detected	Trigger logic (illustrative)	Recommendation
Salary jump	salary growth greater than 30% over 3 months	Premium card, investment SIP, tax-saving FD
New earner	first-ever salary credit	Zero-balance savings, starter SIP, basic insurance
Education expense	recurring school/college-fee payments	Education loan, child insurance plan
Potential home buyer	rent payments detected, no home loan	Home loan pre-approval, home insurance
Health concern	medical spend up more than 50% (3-month window)	Health insurance, never a loan
Financial stress	declining balance + missed EMI	No product push; routed to Sub-Problem 3

Per-Product Model Layer

Product	Model	Key signal	Why
Personal loan	LightGBM (propensity)	EMI headroom, salary stability	Tabular data, needs SHAP explainability, fast to fit
Insurance	LightGBM (separate target)	Life-stage signal, dependents, coverage gap	Distinct label from loan; weights not reused
Investment	Rule-assisted scoring, not ML-first	Savings surplus after EMI + essentials	Mis-selling risk; transparent rules easier to defend than a black box
Credit card	LightGBM	Spend volume, repayment discipline	Reuses loan features with a different target label

Ranking Formula

```
Product_Score = w1·Relevance(segment, product) + w2·LifeStage_Match(signals, product)
+ w3·Financial_Fit(finances, product) + w4·Timing_Score(trigger_recency)
+ w5·Intent_Score(app_behaviour) - Ethical_Penalty(stress, product_type) (hard, non-negotiable)
```

Financial fit caps loan recommendations at debt-to-income below 50%, checks surplus for investment minimums, and measures coverage gap for insurance. Timing score decays exponentially from the trigger event (about 50% decay in 14 days) so stale life-events stop influencing delivery. Only the top 1 to 2 ranked products clear the minimum-score threshold and are surfaced, one at a time.

Build Priority

Loan and insurance propensity models first (highest feature overlap with the stress veto and the clearest explainability story); investment (rules-based) and credit-card models are extensions built only if hackathon time remains, though both stay visible in the architecture diagram as "designed for".

4 . Sub-Problem 2: Vernacular Conversational AI

Approach. The naive translate-input, retrieve-in-English, translate-output pipeline was rejected: it compounds translation error at three points and is unreliable on financial terminology with no clean cross-lingual equivalent. We instead use **multilingual embeddings for retrieval**, avoiding translation as a mandatory step, and split the chatbot into two engines behind one interface, coordinated by a lightweight intent router, because RAG answers policy questions well but cannot reliably drive a structured process like KYC or loan-application form-filling.

Engine	Handles	Example
Slot-Filling (task dialogue)	Structured processes: KYC, loan application, onboarding	"What's your PAN number?" then validate then next field
Multilingual RAG	Open-ended policy / explanation questions	"Why do you need my PAN?" / "What is a cooling-off period?"

Slot-Filling Engine

- **Deterministic validation, not LLM judgment:** PAN regex, Aadhaar checksum (Verhoeff algorithm; full number never stored or logged), income-range sanity checks, loan-amount eligibility caps.
- **Dialogue state tracker:** tracks per-journey slot schema (name, PAN, Aadhaar last 4, income, loan amount, employment type), current slot and retry count; state survives a mid-flow detour into a RAG question.
- **Plain-language correction:** on validation failure, explains the fix in the user's language (for example, "PAN should be 10 characters, like ABCDE1234F") instead of a generic "invalid input".
- **Cross-link to Sub-Problem 1:** a loan amount entered here passes through the same affordability/stress veto layer used by the recommendation engine before the application can proceed: one veto, two entry points.

Multilingual RAG Engine

- **Embedding model:** multilingual-e5 or LaBSE, embedding vernacular queries directly into the document vector space, removing the translate-then-search failure point.
- **Corpus:** RBI financial-awareness messages, PMJDY materials, NPCI/UPI guides, bank product T&Cs; language coverage per source is verified, not assumed. Where a regional version is missing, IndicTrans2 pre-translates once at ingestion and is QC'd.

- **Generation instruction:** the model answers in the detected input language, cites the source document, and explicitly declines to guess if the retrieved context lacks the answer. A hallucinated loan-terms answer is a regulatory liability, not just a UX bug.
- **Intent router:** decides per-turn (not per-session) whether an utterance is task- or knowledge-intent, so a user can detour mid-KYC to ask a policy question and resume the form without losing state.

Key Delivery Decisions

Decision	Choice	Rationale
Primary channel	WhatsApp Business API	500M+ Indian users; lowest onboarding friction for Tier 2/3/4 users
ASR / TTS	AI4Bharat Vakyansh / Indic-TTS, or Google Cloud Speech	9+ Indian languages, over 90% accuracy; natural Indic voices with SSML support
Code-switching	Roman-script transliteration layer + mixed-language NLU	Many users type Hindi in Roman script (Hinglish)
Fallback	Escalate to human agent when NLU confidence is below 0.6	Protects users from being stuck with a misunderstanding bot

Build Priority (Hackathon Scope)

- One journey for the demo: loan application, since it links directly into the veto layer.
- Rule-based / keyword intent router (for example “kya / why / matlab” routes to RAG intent) rather than a trained classifier, given time constraints.
- RAG over a small subset (5 to 10 documents) of RBI/PMJDY sources: enough to demonstrate grounding and citation.
- Voice (ASR/TTS) and the IndicTrans2 QC pass are stretch goals, added only once the text-only vernacular flow is solid.

5 . Sub-Problem 3: Fraud and Financial Stress Detection

Approach. Fraud and financial stress are treated as two distinct detection problems with different math, different data patterns and different interventions, not conflated into one anomaly score.

Dimension	Fraud Detection	Financial Stress Detection
Pattern type	Anomalous, high-deviation, often single-event	Gradual, trend-based, multi-week deterioration
Baseline	Deviation from the user's own transaction history	Deviation from the user's own historical trend line
Model	Isolation Forest / PyOD (unsupervised anomaly detection)	Rolling-window regression / trend-slope classifier
Intervention	Empathetic hold and verification, not auto-decline	Restructuring/counselling offer; suppress new credit pushes

Fraud Detection

Isolation Forest / PyOD (which wraps multiple algorithms: Isolation Forest, AutoEncoder, LOF, behind one API) on the **PaySim** (Kaggle) synthetic mobile-money dataset, purpose-built for fraud simulation. Deep-learning anomaly detection (autoencoders, LSTMs) is deliberately deferred unless the team has prior experience: the marginal accuracy gain over Isolation Forest is unlikely to be visible in a demo, while debugging risk is high.

Financial Stress Detection

Stress signals are **trend-based, not point-anomaly-based**, and are computed as a separate feature set:

- EMI-to-inflow ratio trend, rising over a 3 to 6 month rolling window (not a single spike)
- Savings-rate decay: share of salary retained as balance after 30 days, trending down
- Essential-vs-discretionary spend shift toward essentials (groceries, utilities): belt-tightening
- Balance-floor trend: minimum balance per cycle drifting toward zero across cycles
- Missed/delayed-but-eventually-paid EMI pattern (distinct from a single miss, which is closer to a fraud/error signal)
- Credit-utilisation creep: rising utilisation even while payments stay on-time

Isolation Forest is deliberately not used here: stress is a trend, not an outlier, and an anomaly model flagging a slow decline is a question the architecture should not have to field from a judge.

Financial Health Score (0 to 100, updated daily): Dashboard Surface

Score	Category	Action
80-100	Healthy	Recommend growth products (investment, insurance)
60-79	Watch	Monitor closely; send budgeting tips, savings nudges
40-59	Early Stress	Proactive outreach with budgeting tips; offer EMI restructuring
20-39	Distressed	Connect with counsellor; offer moratorium; pause all product pushes
0-19	Critical	Fraud check; immediate human intervention; regulatory reporting if needed

Cross-Cutting Veto / Decision Layer

A pure, continuously-tunable soft weight was rejected as the sole mechanism: without a guaranteed floor, a high enough base propensity score could theoretically survive the discount and still reach a severely stressed customer, precisely the predatory-nudging failure mode the problem statement warns against. The system instead hybridises soft weighting across the healthy-to-mild-stress range with a **hard rule-based veto** once a defined severity threshold is crossed:

```
stress_score = f(EMI_trend, savings_decay, spend_shift, balance_floor)
if stress_score > HARD_THRESHOLD:
    credit_products = BLOCKED # hard rule, non-negotiable
    eligible_actions = [counselling_nudge, EMI_restructure_offer]
else:
    propensity_score *= (1 - soft_weight(stress_score)) # continuous discount
```

This gives a defensible answer to the hardest question a judge can ask: “*what is the maximum loan-propensity score a severely stressed user can be shown?*” The answer is a fixed, auditable floor, not “it depends on the weight.” Example tone: instead of “You’re in financial trouble,” the system offers help proactively: “*We noticed your balance is lower than usual. Would you like to reschedule your EMI to a later date? No penalty.*”

6 . Ethical Safeguards and Regulatory Compliance

Regulation	Requirement	Our implementation
DPDP Act 2023	Explicit consent, purpose limitation, data minimisation, right to erasure	Tiered consent model (Sec. 3); purpose-bound access; anonymised features; delete-on-request
RBI Data Localization	All financial data resident in India	India-only cloud regions (AWS Mumbai / Azure Pune / GCP Mumbai)
RBI Fair Practices Code	Transparent lending, no predatory practices	Affordability check before any loan recommendation; clear vernacular T&C
Account Aggregator Framework	Data sharing only with explicit consent via AA	Integrated with Setu/Sahamati AA; consent artefacts stored
RBI Digital Lending Guidelines	Full disclosure, cooling-off period, no auto-disbursal	All costs upfront; 3-day cooling-off period implemented

Guardrails Against Predatory Nudging

- **Hard veto floor** beneath the soft-weighting layer, not a discount-only mechanism (Sec. 5).
- **Debt-to-income cap**: a loan is never recommended if it would push DTI above 50%.
- **Single recommendation at a time**, countering the multi-offer pop-up pattern the problem statement names as today's failure mode.
- **Frequency capping**: at most 2 product recommendations per week; no repeat push for a rejected product within 30 days.
- **Transparent rules for investment**, not a black box, given the regulatory sensitivity of investment mis-selling.
- **Opt-out at every touchpoint**: recommendations stop immediately on request.

Algorithmic Bias

Features that function as demographic proxies in Indian contexts (for example, pincode as a caste/religion proxy in some regions) are deliberately excluded even where they might statistically improve accuracy. Bias audits run across gender, geography, income and language, with a disparate-impact monitoring dashboard and human-in-the-loop review for edge cases. The audit explicitly lists excluded features and the reasoning rather than simply asserting the system is “unbiased.”

Explainability by Construction

SHAP values attach to every propensity-model output; RAG answers cite their source document; the veto layer's hard-threshold behaviour is deterministic and auditable rather than emergent from model weights. Any automated action can be overridden by a human relationship manager: the AI assists, it does not decide unilaterally.

7 . Application Security

Because the prototype handles live financial data, the following secure-coding and architecture practices are treated as non-negotiable baseline requirements, not stretch goals.

Secure Coding Practices

- **Input validation and sanitisation:** all user input is treated as untrusted; parameterised queries / ORMs prevent SQL injection, and output is escaped to prevent XSS.
- **API rate limiting:** strict per-endpoint limits so brute-force login attempts or bulk transaction-scraping trigger a temporary IP block rather than succeeding.
- **Zero-trust between microservices:** the AI engine, if deployed as a separate service from the core backend, never blindly trusts the backend's calls; mutual TLS (mTLS) or internal API keys authenticate service-to-service traffic.

Preventing Unauthorised Access

- **IDOR prevention:** every object-level API call (for example, fetching salary or transaction details) verifies that the authenticated user owns the requested record, so "User A" can never view "User B's" balance or history by manipulating an ID in the request, the most common way poorly-built APIs leak data.
- **Multi-factor authentication:** enforced for login and for sensitive actions such as viewing full statements or applying for a loan (OTP via SMS/email or an authenticator app).
- **Robust session management:** short-lived JWT access tokens (about 15-minute expiry) with longer-lived refresh tokens; strict inactivity timeouts (about 5 minutes); tokens stored in HttpOnly, Secure cookies in any web surface to blunt token theft via XSS.

Data Storage and Architecture Security

- Compliance with **RBI data-localisation norms:** financial data resident in India-based regions, consistent with Section 6's regulatory matrix.
- **Encryption at rest and in transit** for all customer and transaction data.
- **Tokenisation** for payment data, so raw account/card numbers are never persisted in plaintext across services.

These practices intersect directly with the DPDP/RBI compliance posture in Section 6: data-localisation and encryption satisfy both a regulatory requirement and a security control, and the consent-tiered data model (Section 3) reduces the attack surface by minimising what any single service can access in the first place.

8 . Technology Stack

Layer	Technology	Purpose
Segmentation	scikit-learn (K-Means / HDBSCAN)	Behavioural customer segmentation
Propensity models	LightGBM	Per-product recommendation scoring; native categorical support, fast on tabular data
Explainability	SHAP	Per-recommendation plain-language justification
Fraud detection	PyOD / Isolation Forest	Unsupervised anomaly detection on transaction data
Stress detection	Rolling-window / logistic regression on trend-slope features	Trend-based financial-distress detection
Vernacular embeddings	multilingual-e5 or LaBSE	Cross-lingual retrieval without mandatory translation
Translation (fallback only)	IndicTrans2 (AI4Bharat)	Offline pre-translation / QC pass for RAG corpus, only where needed
RAG vector store	FAISS (local, hackathon scope)	Document retrieval; Pinecone/Weaviate is the noted production path

Layer	Technology	Purpose
Data pipeline	Apache Kafka (streaming), Apache Spark (batch)	Real-time ingestion + batch feature refresh
Prototype UI	Streamlit	Fast interactive dashboard; renders SHAP plots natively
API layer	FastAPI	Thin service layer between models and UI/frontend
Production frontend (stretch)	Next.js + Tailwind	Only after the backend/model pipeline is functional and demoable
Cloud	AWS Mumbai / Azure Pune (RBI-compliant regions)	Data residency, elastic scaling
Monitoring	Grafana + Prometheus + Evidently AI	Model and infrastructure observability, drift detection

9 . MVP Scope and Build Roadmap

Component	Hackathon MVP	Full version
Languages	Hindi + English	10+ Indian languages
Channel	Web-based chatbot demo	WhatsApp, App, IVR, USSD
Recommendation	Segmentation + rules on synthetic data	Full LightGBM pipeline on real CBS data
Stress/fraud detection	Threshold rules + Isolation Forest on PaySim	Ensemble ML on live streams
Data	Synthetic dataset (about 100 customers, 6 months) + PaySim	Real CBS integration via APIs
Explainability	Template-based explanations	Full SHAP-based dynamic explanations

Build Order

Step	Task
1	Shared feature-engineering pipeline + stress/fraud veto layer
2	Loan propensity model + SHAP explanation
3	Insurance propensity model
4	Fraud detection (Isolation Forest / PyOD) on PaySim
5	Stress-trend detection model
6	Streamlit dashboard wired to steps 2-5, live veto behaviour demoable
7	Slot-filling engine for the loan-application journey, linked to the veto layer
8	Multilingual RAG over 5-10 RBI/PMJDY documents + rule-based intent router
9 (stretch)	Investment rules engine, credit-card model
10 (stretch)	Next.js frontend, voice ASR/TTS, IndicTrans2 QC pass

10 . Judging Criteria Alignment and Honest Gaps

Criterion	How this system addresses it
Innovation and technical feasibility	Multi-specialist-model architecture on one shared pipeline; feasible in hackathon time with well-documented libraries (LightGBM, PyOD, FAISS)
Depth of personalization vs. genuine benefit	Hard veto floor blocks credit pushes to stressed users; single-recommendation-at-a-time; transparent rules for investment to avoid mis-selling

Criterion	How this system addresses it
Explainability and RBI/regulatory readiness	SHAP on every recommendation; RAG citations; deterministic hard-threshold logic; DPDP-aligned consent-tiered data model
Usability for non-tech-savvy, vernacular-first users	Multilingual embedding retrieval (no fragile translate step); plain-language slot-filling error correction; native-language generation
Scalability across existing infrastructure	FastAPI overlay sits alongside existing bank APIs (Finacle/Flexcube/TCS BaNCS); FAISS/Streamlit has a clear path to Pinecone/Weaviate + Next.js without architectural rework

Known Limitations and Honest Gaps

- No single public dataset covers all three sub-problems; PaySim, Bank Marketing and Lending Club data are stitched together and this simplification is disclosed rather than hidden.
- Stress-severity thresholds (percentile-based vs. fixed domain thresholds, for example EMI ratio above 50%) are not yet finalised: a build-time decision with direct compliance implications.
- Regional-language document coverage for the RAG corpus is not yet verified and may require offline pre-translation for some languages.
- Voice interaction (ASR/TTS) is out of scope for the initial build and remains a stretch goal.
- The chatbot demo covers a single journey (loan application) rather than full KYC/onboarding; the slot-filling architecture generalises to more journeys but this is not yet built.
- Credit-card and investment models are lower priority and may remain at the design/architecture stage depending on remaining build time.

BharatBanker AI: AI-Powered Hyper-Personalized Banking for Bharat. This document is for hackathon evaluation purposes only.